Team 3 Zip Code Project 3.0

Generated by Doxygen 1.14.0

1 Class Index 1
1.1 Class List . . . 1

2 File Index 3
2.1 File List . . . 3

3 Class Documentation 5
3.1 Analyzer Class Reference . . . 5
3.1.1 Member Function Documentation . . . 5
3.1.1.1 consume() . . . 5
3.1.1.2 getResults() . . . 6
3.2 Block Class Reference . . . 6
3.2.1 Constructor & Destructor Documentation . . . 7
3.2.1.1 Block() . . . 7
3.2.2 Member Function Documentation . . . 8
3.2.2.1 AddRecord() . . . 8
3.2.2.2 GetByteMaxSize() . . . 8
3.2.2.3 GetByteMinSize() . . . 8
3.2.2.4 GetByteSize() . . . 9
3.2.2.5 GetNextRBN() . . . 9
3.2.2.6 GetPrevRBN() . . . 9
3.2.2.7 GetRBN() . . . 9
3.2.2.8 GetRecordCount() . . . 10
3.2.2.9 GetRecords() . . . 10
3.2.2.10 ReadBlock() . . . 10
3.2.2.11 RemoveRecordAt() . . . 10
3.2.2.12 SetNextRBN() . . . 11
3.2.2.13 SetPrevRBN() . . . 11
3.2.2.14 WriteBlock() . . . 12
3.3 BlockBuffer Class Reference . . . 12
3.3.1 Detailed Description . . . 13
3.3.2 Constructor & Destructor Documentation . . . 13
3.3.2.1 BlockBuffer() . . . 13
3.3.3 Member Function Documentation . . . 13
3.3.3.1 Clear() . . . 13
3.3.3.2 Init() . . . 13
3.3.3.3 Pack() . . . 14
3.3.3.4 Print() . . . 14
3.3.3.5 Read() . . . 15
3.3.3.6 Unpack() . . . 15
3.3.3.7 UnpackFieldsToRecord() . . . 16
3.3.3.8 UnpackRecordFromBlock() [1/2] . . . 16
3.3.3.9 UnpackRecordFromBlock() [2/2] . . . 17

3.3.3.10 Write() . . . 17
3.4 BlockedSequence Class Reference . . . 18
3.4.1 Constructor & Destructor Documentation . . . 19
3.4.1.1 BlockedSequence() . . . 19
3.4.2 Member Function Documentation . . . 19
3.4.2.1 AppendRecord() . . . 19
3.4.2.2 Delete() . . . 20
3.4.2.3 DumpLogicalBlockOrder() . . . 20
3.4.2.4 DumpLogicalOrder() . . . 21
3.4.2.5 DumpPhysicalBlockOrder() . . . 21
3.4.2.6 GetAvailListSize() . . . 21
3.4.2.7 GetBlock() . . . 21
3.4.2.8 GetCount() . . . 22
3.4.2.9 GetHeadRBN() . . . 22
3.4.2.10 GetNextRBN() . . . 22
3.4.2.11 GetTailRBN() . . . 23
3.4.2.12 Insert() . . . 23
3.4.2.13 IsEmpty() . . . 23
3.4.2.14 LoadAllFromHead() . . . 24
3.4.2.15 WriteAll() . . . 24
3.5 CsvToLength Class Reference . . . 24
3.5.1 Constructor & Destructor Documentation . . . 24
3.5.1.1 CsvToLength() . . . 24
3.6 HeaderBuffer Class Reference . . . 25
3.6.1 Member Function Documentation . . . 25
3.6.1.1 readHeader() . . . 25
3.6.1.2 writeHeader() . . . 26
3.7 HeaderRecord Class Reference . . . 26
3.7.1 Detailed Description . . . 27
3.7.2 Constructor & Destructor Documentation . . . 27
3.7.2.1 HeaderRecord() [1/2] . . . 27
3.7.2.2 HeaderRecord() [2/2] . . . 27
3.7.3 Member Function Documentation . . . 28
3.7.3.1 ConfigureBlockedSequenceHeader() . . . 28
3.7.3.2 GetHeader() . . . 28
3.7.3.3 ReadHeader() . . . 28
3.7.3.4 WriteHeader() . . . 29
3.7.4 Friends And Related Symbol Documentation . . . 29
3.7.4.1 HeaderBuffer . . . 29
3.8 Length_Indicated_ZipCodeBuffer Class Reference . . . 29
3.8.1 Detailed Description . . . 30
3.8.2 Constructor & Destructor Documentation . . . 30

3.8.2.1 Length_Indicated_ZipCodeBuffer() . . . 30
3.8.2.2 ~Length_Indicated_ZipCodeBuffer() . . . 30
3.8.3 Member Function Documentation . . . 31
3.8.3.1 packRecord() . . . 31
3.8.3.2 readNext() . . . 31
3.9 PrimaryKeyIndex Class Reference . . . 32
3.9.1 Constructor & Destructor Documentation . . . 32
3.9.1.1 PrimaryKeyIndex() . . . 32
3.9.2 Member Function Documentation . . . 32
3.9.2.1 BuildFromBlocks() . . . 32
3.9.2.2 Dump() . . . 33
3.9.2.3 find() . . . 33
3.9.2.4 LoadFromIndexFile() . . . 33
3.9.2.5 ReadFromFile() . . . 34
3.9.2.6 SetBlockDirectory() . . . 34
3.9.2.7 WriteToFile() . . . 34
3.10 Printer Class Reference . . . 35
3.10.1 Detailed Description . . . 35
3.10.2 Member Function Documentation . . . 35
3.10.2.1 printTable() . . . 35
3.11 PrinterCSV Class Reference . . . 36
3.11.1 Detailed Description . . . 36
3.11.2 Member Function Documentation . . . 36
3.11.2.1 printCSV() . . . 36
3.12 RecordFormatter Class Reference . . . 37
3.12.1 Detailed Description . . . 37
3.12.2 Member Function Documentation . . . 37
3.12.2.1 formatLabeled() . . . 37
3.12.2.2 pack() . . . 38
3.12.2.3 unpack() . . . 38
3.13 StateExtremes Struct Reference . . . 38
3.13.1 Detailed Description . . . 38
3.13.2 Member Data Documentation . . . 39
3.13.2.1 easternmost . . . 39
3.13.2.2 initialized . . . 39
3.13.2.3 northernmost . . . 39
3.13.2.4 southernmost . . . 39
3.13.2.5 westernmost . . . 39
3.14 ZipCodeBuffer Class Reference . . . 39
3.14.1 Detailed Description . . . 40
3.14.2 Constructor & Destructor Documentation . . . 40
3.14.2.1 ZipCodeBuffer() . . . 40

3.14.2.2 ~ZipCodeBuffer() . . . 40
3.14.3 Member Function Documentation . . . 40
3.14.3.1 readNext() . . . 40
3.15 ZipCodeRecord Class Reference . . . 41
3.15.1 Constructor & Destructor Documentation . . . 42
3.15.1.1 ZipCodeRecord() [1/2] . . . 42
3.15.1.2 ZipCodeRecord() [2/2] . . . 42
3.15.2 Member Function Documentation . . . 42
3.15.2.1 getLatitude() . . . 42
3.15.2.2 getLongitude() . . . 43
3.15.2.3 getState() . . . 43
3.15.2.4 getZip() . . . 43

4 File Documentation **45**
4.1 include/analyzer.h File Reference . . . 45
4.1.1 Detailed Description . . . 45
4.2 analyzer.h . . . 46
4.3 include/CsvToLength.h File Reference . . . 46
4.3.1 Detailed Description . . . 46
4.4 CsvToLength.h . . . 47
4.5 include/HeaderRecord.h File Reference . . . 47
4.5.1 Detailed Description . . . 47
4.6 HeaderRecord.h . . . 48
4.7 include/Length_Indicated_ZipCodeBuffer.h File Reference . . . 49
4.7.1 Detailed Description . . . 49
4.8 Length_Indicated_ZipCodeBuffer.h . . . 49
4.9 include/PrimaryKeyIndex.h File Reference . . . 50
4.9.1 Detailed Description . . . 50
4.10 PrimaryKeyIndex.h . . . 50
4.11 include/Printer.h File Reference . . . 51
4.11.1 Detailed Description . . . 51
4.12 Printer.h . . . 52
4.13 include/PrinterCSV.h File Reference . . . 52
4.13.1 Detailed Description . . . 52
4.14 PrinterCSV.h . . . 53
4.15 include/Project 3.0/Block.h File Reference . . . 53
4.15.1 Detailed Description . . . 53
4.16 Block.h . . . 54
4.17 include/Project 3.0/BlockBuffer.h File Reference . . . 54
4.17.1 Detailed Description . . . 55
4.18 BlockBuffer.h . . . 55
4.19 include/Project 3.0/BlockedSequence.h File Reference . . . 56

4.19.1 Detailed Description . . . 56
4.20 BlockedSequence.h . . . 57
4.21 include/Project 3.0/HeaderBuffer.h File Reference . . . 57
4.21.1 Detailed Description . . . 58
4.22 HeaderBuffer.h . . . 58
4.23 include/RecordFormatter.h File Reference . . . 58
4.23.1 Detailed Description . . . 59
4.24 RecordFormatter.h . . . 59
4.25 include/StateExtremes.h File Reference . . . 59
4.25.1 Detailed Description . . . 60
4.26 StateExtremes.h . . . 60
4.27 include/ZipCodeBuffer.h File Reference . . . 61
4.27.1 Detailed Description . . . 61
4.28 ZipCodeBuffer.h . . . 62
4.29 include/ZipCodeRecord.h File Reference . . . 62
4.29.1 Detailed Description . . . 62
4.30 ZipCodeRecord.h . . . 63
4.31 src/analyzer.cpp File Reference . . . 63
4.31.1 Detailed Description . . . 63
4.32 src/CsvToLength.cpp File Reference . . . 64
4.32.1 Detailed Description . . . 64
4.32.2 Macro Definition Documentation . . . 64
4.32.2.1 CSVTOLENGTH_CPP . . . 64
4.33 src/driver_csv_to_length_indicated.cpp File Reference . . . 64
4.33.1 Detailed Description . . . 65
4.33.2 Function Documentation . . . 65
4.33.2.1 main() . . . 65
4.34 src/HeaderRecord.cpp File Reference . . . 65
4.34.1 Detailed Description . . . 66
4.34.2 Macro Definition Documentation . . . 66
4.34.2.1 HEADERRECORD_CPP . . . 66
4.34.3 Function Documentation . . . 66
4.34.3.1 operator+() [1/2] . . . 66
4.34.3.2 operator+() [2/2] . . . 66
4.35 src/Length_Indicated_ZipCodeBuffer.cpp File Reference . . . 66
4.35.1 Detailed Description . . . 67
4.36 src/main.cpp File Reference . . . 67
4.36.1 Detailed Description . . . 67
4.36.2 Function Documentation . . . 67
4.36.2.1 main() . . . 67
4.37 src/PrimaryKeyIndex.cpp File Reference . . . 68
4.37.1 Detailed Description . . . 68

4.37.2 Macro Definition Documentation . . . 68
4.37.2.1 PRIMARYKEYINDEX_CPP . . . 68
4.38 src/printer.cpp File Reference . . . 69
4.38.1 Detailed Description . . . 69
4.38.2 Macro Definition Documentation . . . 69
4.38.2.1 PRINTER_CPP . . . 69
4.39 src/printerCSV.cpp File Reference . . . 69
4.39.1 Detailed Description . . . 70
4.39.2 Macro Definition Documentation . . . 70
4.39.2.1 PRINTERCSV_CPP . . . 70
4.40 src/Project 3.0/Block.cpp File Reference . . . 70
4.40.1 Detailed Description . . . 70
4.40.2 Macro Definition Documentation . . . 71
4.40.2.1 BLOCK_CPP . . . 71
4.41 src/Project 3.0/BlockBuffer.cpp File Reference . . . 71
4.41.1 Detailed Description . . . 71
4.42 src/Project 3.0/BlockedSequence.cpp File Reference . . . 71
4.42.1 Detailed Description . . . 72
4.42.2 Macro Definition Documentation . . . 72
4.42.2.1 BLOCKEDSEQUENCE_CPP . . . 72
4.43 src/Project 3.0/HeaderBuffer.cpp File Reference . . . 72
4.43.1 Detailed Description . . . 72
4.43.2 Macro Definition Documentation . . . 73
4.43.2.1 HEADBUFFER_CPP . . . 73
4.44 src/Project 3.0/TestDriver.cpp File Reference . . . 73
4.44.1 Detailed Description . . . 73
4.44.2 Function Documentation . . . 73
4.44.2.1 main() . . . 73
4.45 src/RecordFormatter.cpp File Reference . . . 74
4.45.1 Detailed Description . . . 74
4.46 src/SearchIndex.cpp File Reference . . . 74
4.46.1 Detailed Description . . . 74
4.46.2 Function Documentation . . . 74
4.46.2.1 main() . . . 74
4.47 src/ZipCodeBuffer.cpp File Reference . . . 75
4.47.1 Detailed Description . . . 75
4.48 src/ZipCodeRecord.cpp File Reference . . . 75
4.48.1 Detailed Description . . . 75

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Analyzer 5
Block 6
BlockBuffer
Buffer helper for reading/writing packed fields and unpacking records from blocks 12
BlockedSequence 18
CsvToLength 24
HeaderBuffer 25
HeaderRecord
Metadata model for length-indicated and blocked-sequence file headers 26
Length_Indicated_ZipCodeBuffer
Provides seqential access to zip code records from a data source 29
PrimaryKeyIndex 32
Printer
Responsible for printing the results of the analysis in a formatted table. It provides a method to print the table of state extremes, which includes the easternmost, ... 35
PrinterCSV
The Printer class is responsible for printing the results of the analysis in a CSV file. It provides a method to print the table of state extremes, which includes the easternmost, ... 36
RecordFormatter
Methods for packing and unpacking ZipCodeRecord objects into a length-indicated format. It allows for efficient serialization and deserialization of ZipCodeRecord data, making it easier to store and transmit records in a compact format 37
StateExtremes 38
ZipCodeBuffer
Provides seqential access to zip code records from a data source 39
ZipCodeRecord 41

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

include/analyzer.h
Header file for the Analyzer class, which provides methods for analyzing data sets 45
include/CsvToLength.h
Declares CsvToLength, a utility class that converts a CSV file into a length-indicated output file 46
include/HeaderRecord.h
Declares the HeaderRecord class used to build metadata for a length-indicated data file 47
include/Length_Indicated_ZipCodeBuffer.h
Declares the LengthIndicatedZipCodeBuffer class, which reads and writes length indicated Zip Code records 49
include/PrimaryKeyIndex.h
Declares PrimaryKeyIndex, a primary key index that can be used to display the Zip Code data for all Zip Codes listed on the command line 50
include/Printer.h
Header file for the Printer class, which provides methods for printing the results of the analysis 51
include/PrinterCSV.h
Header file for the PrinterCSV class, which provides methods for printing the results of the analysis into a CSV file 52
include/RecordFormatter.h
Declares the RecordFormatter class, which provides methods for packing and unpacking ZipCodeRecord objects into a length-indicated format 58
include/StateExtremes.h
Header file for the StateExtremes class, which provides methods for analyzing data sets 59
include/ZipCodeBuffer.h
This declares the ZipCodeBuffer class, which is responsible for buffering and managing zip code data for efficient access and processing 61
include/ZipCodeRecord.h
Header file for the ZipCodeRecord class, which provides a data structure for storing zip code information 62
include/Project 3.0/Block.h
Declares the Block container used to store packed records within a fixed byte-capacity unit . . 53
include/Project 3.0/BlockBuffer.h
Declares BlockBuffer utilities for packing/unpacking block and record data 54
include/Project 3.0/BlockedSequence.h
Declares the BlockedSequence container used to manage linked blocks by RBN 56
include/Project 3.0/HeaderBuffer.h
Declares the HeaderBuffer class for reading and writing header records 57

src/analyzer.cpp
Implementation of the Analyzer class for data processing 63
src/CsvToLength.cpp
Implements CSV-to-length-indicated conversion logic 64
src/driver_csv_to_length_indicated.cpp
Driver for csv to length indicated file 64
src/HeaderRecord.cpp
Implements HeaderRecord, which builds a serialized header record for the length-indicated file format 65
src/Length_Indicated_ZipCodeBuffer.cpp
Implementation file for the Length_Indicated_ZipCodeBuffer class, which provides methods for reading and writing length-indicated Zip Code records 66
src/main.cpp
Production CLI entry point for Project 3.0 blocked-sequence generation 67
src/PrimaryKeyIndex.cpp
Implements a primary key index that can be used to display the Zip Code data for all Zip Codes listed on the command line 68
src/printer.cpp
Implementation file for the Printer class, which provides methods for printing the results of the analysis 69
src/printerCSV.cpp
Implementation file for the PrinterCSV class, which provides methods for printing the results of the analysis 69
src/RecordFormatter.cpp
Implements the RecordFormatter class, which provides methods for packing and unpacking ZipCode 74
src/SearchIndex.cpp
Search program for blocked-sequence data using a simple primary-key index 74
src/ZipCodeBuffer.cpp
Implementation file for the ZipCodeBuffer class, which provides methods for reading and managing zip code data 75
src/ZipCodeRecord.cpp
Implementation file for the ZipCodeRecord class, which provides a data structure for storing zip code information 75
src/Project 3.0/Block.cpp
Implements Block behavior for record storage, metadata updates, and capacity checks 70
src/Project 3.0/BlockBuffer.cpp
Implementation file for the BlockBuffer class which unpacks a record from a block into a record buffer. Reads/writes whole fixed-size blocks (default 512 bytes) 71
src/Project 3.0/BlockedSequence.cpp
Implements BlockedSequence operations for a doubly-linked block chain 71
src/Project 3.0/HeaderBuffer.cpp
Implements HeaderBuffer methods for header-record serialization 72
src/Project 3.0/TestDriver.cpp
Minimal scenario runner for Project 3.0 73

Chapter 3

Class Documentation

3.1 Analyzer Class Reference

```
#include <analyzer.h>
```

Public Member Functions

- void consume (const ZipCodeRecord &record)

  consumes one ZipCodeRecord and updates per state extremes.
- const std::map< std::string, StateExtremes > & getResults () const

  Returns the results of ComputeExtremes.

3.1.1 Member Function Documentation

3.1.1.1 consume()

```
void Analyzer::consume (
            const ZipCodeRecord & record)
```

consumes one ZipCodeRecord and updates per state extremes.

consumed ZipCodeRecord objects one at a time and updates per extreme values using coordinate comparisons

Parameters

	record	Parsed CSV row

The consume function takes a ZipCodeRecord object as input and updates the per state.

Returns

void

Parameters

	record	Parsed CSV row as a ZipCodeRecord object

The consume function takes a ZipCodeRecord object as input and updates the per state.

Returns

void

3.1.1.2 getResults()

```
const std::map< std::string, StateExtremes > & Analyzer::getResults () const
```

Returns the results of ComputeExtremes.

returns the map of state extremes, which is a map from state ID to that state's extremes.

Parameters

	State	The state for which to return the extremes

The map is returned as a const reference to avoid unnecessary copying of the data, and const reference to ensure that the caller cannot modify the internal state of the Analyzer class, while still allowing them to access the results of the analysis.

Returns

A const reference to the map of state extremes, which is a map from state ID to that state's extremes.

The map of state extremes as a const reference to prevent modification of the internal state of the Analyzer class.

The map is returned as a const reference to avoid unnecessary copying of the data, which can be expensive in terms of performance. By returning a const reference, we ensure that the caller cannot modify the internal state of the Analyzer class, while still allowing them to access the results of the analysis.

The documentation for this class was generated from the following files:

- include/analyzer.h
- src/analyzer.cpp

3.2 Block Class Reference

```
#include <Block.h>
```

Public Member Functions

- Block (int prevrbn, int rbn)

  Constructs an empty block and initializes link metadata.
- bool AddRecord (const string &Record)

  Attempts to append one packed record into this block.
- bool RemoveRecordAt (size_t recordIndex)

  Removes a record by index and refreshes block accounting metadata.
- bool ReadBlock ()

  Reads this block's serialized representation from storage.
- bool WriteBlock ()

  Writes this block's serialized representation to storage.
- int GetRecordCount ()

  Returns the number of records stored in this block.
- int GetRBN ()

  Returns this block's RBN.
- int GetPrevRBN ()

  Returns the previous block's RBN.
- int GetNextRBN ()

  Returns the next block's RBN.
- int GetByteSize ()

  Returns total payload bytes currently consumed in the block.
- int GetByteMaxSize ()

  Returns the maximum payload capacity in bytes for this block.
- int GetByteMinSize ()

  Returns the minimum payload capacity in bytes for this block.
- vector< string > & GetRecords ()

  Provides mutable access to stored record payloads.
- void SetNextRBN (int nextRbn)

  Updates the next-block link and refreshes serialized header data.
- void SetPrevRBN (int prevRbn)

  Updates the previous-block link and refreshes serialized header data.

3.2.1 Constructor & Destructor Documentation

3.2.1.1 Block()

```
Block::Block (
            int prevrbn,
            int rbn)
```

Constructs an empty block and initializes link metadata.

Constructs an empty block with initialized links and metadata.

Parameters

	prevrbn	RBN of the previous block in the sequence.
	rbn	RBN assigned to this block.
	prevrbn	RBN of the previous block in the chain.
	rbn	RBN assigned to this block.

3.2.2 Member Function Documentation

3.2.2.1 AddRecord()

```
bool Block::AddRecord (
            const string & Record)
```

Attempts to append one packed record into this block.

Attempts to append one packed record to this block.

Parameters

	Record	Length-indicated record text to add.

Returns

True if the record is valid and fits in the remaining capacity; otherwise false.

Parameters

	Record	Length-indicated record string.

Returns

True if record format is valid and capacity permits insertion; otherwise false.

Expects a leading numeric size prefix before the first comma. Uses that size to enforce block byte-capacity limits.

3.2.2.2 GetByteMaxSize()

```
int Block::GetByteMaxSize ()
```

Returns the maximum payload capacity in bytes for this block.

Returns

Maximum byte capacity.

3.2.2.3 GetByteMinSize()

```
int Block::GetByteMinSize ()
```

Returns the minimum payload capacity in bytes for this block.

Returns

Minimum byte capacity.

3.2.2.4 GetByteSize()

```
int Block::GetByteSize ()
```

Returns total payload bytes currently consumed in the block.

Returns the payload bytes currently used by records.

Returns

Number of used payload bytes.

Current byte usage.

3.2.2.5 GetNextRBN()

```
int Block::GetNextRBN ()
```

Returns the next block's RBN.

Returns the next block link.

Returns

Next block RBN.

3.2.2.6 GetPrevRBN()

```
int Block::GetPrevRBN ()
```

Returns the previous block's RBN.

Returns the previous block link.

Returns

Previous block RBN.

3.2.2.7 GetRBN()

```
int Block::GetRBN ()
```

Returns this block's RBN.

Returns this block's relative block number.

Returns

Current block RBN.

Block RBN.

3.2.2.8 GetRecordCount()

```
int Block::GetRecordCount ()
```

Returns the number of records stored in this block.

Returns

Current record count.

3.2.2.9 GetRecords()

```
vector< string > & Block::GetRecords ()
```

Provides mutable access to stored record payloads.

Returns mutable access to the block's stored records.

Returns

Reference to the internal record vector.

Reference to the record vector.

3.2.2.10 ReadBlock()

```
bool Block::ReadBlock ()
```

Reads this block's serialized representation from storage.

Reads this block's serialized representation from disk.

Returns

True on successful read and parse; otherwise false.

True on successful read and parse; otherwise false.

first line is block metadata, then one record per line.

3.2.2.11 RemoveRecordAt()

```
bool Block::RemoveRecordAt (
            size_t recordIndex)
```

Removes a record by index and refreshes block accounting metadata.

Removes one record from this block by index.

Parameters

	recordIndex	Zero-based record index to remove.

Returns

True if the record exists and is removed; otherwise false.

Parameters

	recordIndex	Zero-based record index.

Returns

True if removal succeeds; otherwise false.

Parses the packed record's size prefix to keep byte accounting consistent with AddRecord.

3.2.2.12 SetNextRBN()

```
void Block::SetNextRBN (
            int nextRbn)
```

Updates the next-block link and refreshes serialized header data.

Updates the next block link and refreshes header metadata.

Parameters

	nextRbn	RBN value to assign as the next block link.
	nextRbn	New next block RBN value.

3.2.2.13 SetPrevRBN()

```
void Block::SetPrevRBN (
            int prevRbn)
```

Updates the previous-block link and refreshes serialized header data.

Updates the previous block link and refreshes header metadata.

Parameters

	prevRbn	RBN value to assign as the previous block link.
	prevRbn	New previous block RBN value.

3.2.2.14 WriteBlock()

```
bool Block::WriteBlock ()
```

Writes this block's serialized representation to storage.

Writes this block's serialized representation to disk.

Returns

True on successful write; otherwise false.

True on successful write; otherwise false.

Uses a simple textbook-style format: first line is block metadata, then one record per line.

The documentation for this class was generated from the following files:

- include/Project 3.0/Block.h
- src/Project 3.0/Block.cpp

3.3 BlockBuffer Class Reference

Buffer helper for reading/writing packed fields and unpacking records from blocks.

```
#include <BlockBuffer.h>
```

Public Member Functions

- BlockBuffer (int maxBytes=512)

  Constructs a block buffer with a maximum byte capacity.
- void Clear ()

  Clears packed content and resets cursor positions.
- int Read (istream &stream)

  Reads packed bytes from a stream into the internal buffer.
- int Write (ostream &stream) const

  Writes packed bytes from the internal buffer to a stream.
- int Pack (const char ∗, short size=-1)

  Packs one field string into the internal buffer.
- int Unpack (char ∗)

  Unpacks one field string from the internal buffer.
- void Print (ostream &) const

  Prints a short diagnostic summary of the internal buffer.
- int Init (int maxBytes=512)

  Initializes or reinitializes internal storage.
- bool UnpackRecordFromBlock (Block &block, int recordIndex, std::string &recordBufferOut) const

  Extracts one packed record string from a block.
- bool UnpackFieldsToRecord (const std::string &recordBuffer, ZipCodeRecord &recordOut) const

  Parses one packed record buffer into a ZipCodeRecord object.
- bool UnpackRecordFromBlock (Block &block, int recordIndex, ZipCodeRecord &recordOut) const

  Convenience method: unpack a block record directly into an object.

3.3.1 Detailed Description

Buffer helper for reading/writing packed fields and unpacking records from blocks.

3.3.2 Constructor & Destructor Documentation

3.3.2.1 BlockBuffer()

```
BlockBuffer::BlockBuffer (
            int maxBytes = 512)
```

Constructs a block buffer with a maximum byte capacity.

Constructs a BlockBuffer with the specified maximum byte capacity.

Parameters

	maxBytes	Maximum number of bytes that can be stored in the internal buffer.
	maxBytes	Maximum number of bytes the buffer can hold.

Allocates the internal byte array and initializes the buffer state.

3.3.3 Member Function Documentation

3.3.3.1 Clear()

```
void BlockBuffer::Clear ()
```

Clears packed content and resets cursor positions.

Clears the buffer state so it can be reused.

Resets the current cursor position and stored size to zero.

3.3.3.2 Init()

```
int BlockBuffer::Init (
            int maxBytes = 512)
```

Initializes or reinitializes internal storage.

Initializes the buffer storage.

Parameters

	maxBytes	Maximum allowed bytes for this buffer.

Returns

Non-zero when initialization succeeds.

Parameters

	maxBytes	Maximum number of bytes the buffer can hold.

Returns

True if initialization succeeded.

Allocates the internal byte array and resets the buffer state.

3.3.3.3 Pack()

```
int BlockBuffer::Pack (
            const char * str,
            short size = -1)
```

Packs one field string into the internal buffer.

Packs a string into the buffer using a length prefix.

Parameters

	str	Input C-string field value.
	size	Optional explicit size, or -1 to use string length.

Returns

Non-zero when pack succeeds; otherwise zero.

Parameters

	str	String to store in the buffer.
	size	Number of bytes to pack, or -1 to use strlen(str).

Returns

True if the string was packed successfully; otherwise false.

Stores the string length first, then copies the character data into the internal buffer and advances the cursor.

3.3.3.4 Print()

```
void BlockBuffer::Print (
            ostream & stream) const
```

Prints a short diagnostic summary of the internal buffer.

Prints a summary of the buffer state.

Parameters

	stream	Output stream.
	stream	Output stream used for display.

3.3.3.5 Read()

```
int BlockBuffer::Read (
            istream & stream)
```

Reads packed bytes from a stream into the internal buffer.

Reads a packed buffer from an input stream.

Parameters

	stream	Source stream.

Returns

Non-zero when read succeeds; otherwise zero.

Parameters

	stream	Input stream containing length-prefixed packed data.

Returns

True if the buffer was read successfully; otherwise false.

Reads the size prefix first, validates the size, and then reads the packed bytes into the internal buffer.

3.3.3.6 Unpack()

```
int BlockBuffer::Unpack (
            char * str)
```

Unpacks one field string from the internal buffer.

Unpacks the next length-prefixed string from the buffer.

Parameters

	str	Destination C-string buffer.

Returns

Non-zero when unpack succeeds; otherwise zero.

Parameters

	str	Destination buffer that receives the unpacked string.

Returns

True if a string was unpacked successfully; otherwise false.

Reads the stored length, copies the matching bytes to the output, and appends a null terminator.

3.3.3.7 UnpackFieldsToRecord()

```
bool BlockBuffer::UnpackFieldsToRecord (
            const std::string & recordBuffer,
            ZipCodeRecord & recordOut) const
```

Parses one packed record buffer into a ZipCodeRecord object.

Parameters

	recordBuffer	Packed record in format <len>,<zip>,<state>,<lat>,<long>.
	recordOut	Output record object.

Returns

true if parse succeeds; otherwise false.

Parameters

	recordBuffer	Packed record in format <len>,<zip>,<state>,<lat>,<long>.
	recordOut	Output ZipCodeRecord populated from parsed fields.

Returns

true when parsing and conversions succeed; otherwise false.

3.3.3.8 UnpackRecordFromBlock() [1/2]

```
bool BlockBuffer::UnpackRecordFromBlock (
            Block & block,
            int recordIndex,
            std::string & recordBufferOut) const
```

Extracts one packed record string from a block.

Copies one packed record from a block into a record-buffer string.

Parameters

block	Source block containing packed records.
recordIndex	Zero-based index of the record in the block.
recordBufferOut	Output packed record string.

Returns

true if extraction succeeds; otherwise false.

Parameters

block	Source block containing packed records.
recordIndex	Zero-based index of the desired record.
recordBufferOut	Output packed record string.

Returns

true when the index is valid and the record is copied; otherwise false.

3.3.3.9 UnpackRecordFromBlock() [2/2]

```
bool BlockBuffer::UnpackRecordFromBlock (
            Block & block,
            int recordIndex,
            ZipCodeRecord & recordOut) const
```

Convenience method: unpack a block record directly into an object.

Convenience overload that performs both unpack stages.

Parameters

block	Source block.
recordIndex	Zero-based index into block records.
recordOut	Output record object.

Returns

true if both unpack stages succeed; otherwise false.

Parameters

block	Source block.
recordIndex	Zero-based record index in the block.
recordOut	Output record object.

Returns

true when both unpack-from-block and parse-to-record succeed.

3.3.3.10 Write()

```
int BlockBuffer::Write (
            ostream & stream) const
```

Writes packed bytes from the internal buffer to a stream.

Writes the current packed buffer to an output stream.

Parameters

	stream	Destination stream.

Returns

Non-zero when write succeeds; otherwise zero.

Parameters

	stream	Output stream that receives the buffer contents.

Returns

True if the buffer was written successfully; otherwise false.

Writes the stored size first, followed by the raw packed bytes.

The documentation for this class was generated from the following files:

- include/Project 3.0/BlockBuffer.h
- src/Project 3.0/BlockBuffer.cpp

3.4 BlockedSequence Class Reference

```
#include <BlockedSequence.h>
```

Public Member Functions

- BlockedSequence (int startRBN=1)

  Constructs an empty blocked sequence.
- bool IsEmpty ()

  Indicates whether the sequence has any blocks.
- int GetHeadRBN ()

  Returns the RBN of the head block.
- int GetTailRBN ()

  Returns the RBN of the tail block.
- int GetNextRBN ()

  Returns the next RBN that will be assigned to a new block.
- int GetCount ()

  Returns the number of blocks currently tracked in memory.
- bool AppendRecord (const string &recordCsv)

  Appends one packed record to the logical tail of the sequence.
- bool WriteAll ()

  Writes every linked block from head to tail.
- bool LoadAllFromHead ()

  Reads every linked block from head to tail.
- Block * GetBlock (int rbn)

Looks up a block by relative block number.

- string DumpLogicalOrder ()

  Dumps the logical sequence of records from head to tail as a string.
- string DumpPhysicalBlockOrder ()

  Dumps active blocks in physical order (ascending RBN map order).
- string DumpLogicalBlockOrder ()

  Dumps active blocks in logical order (following next-RBN chain).
- bool Insert (const string &recordCsv, string &logOutput)

  Inserts a record into the appropriate block, handling block splits if needed.
- bool Delete (const string &zipKey, string &logOutput)

  Deletes a record by ZIP key, handling block merges/redistributions if needed.
- int GetAvailListSize ()

  Returns the number of blocks currently in the available list.

3.4.1 Constructor & Destructor Documentation

3.4.1.1 BlockedSequence()

```
BlockedSequence::BlockedSequence (
            int startRBN = 1)
```

Constructs an empty blocked sequence.

Parameters

	startRBN	Initial RBN value to assign to the first created block.

Initializes head and tail as empty (0) and stores the next RBN counter used for newly allocated blocks.

Parameters

	startRBN	Initial RBN value assigned to the first new block.

3.4.2 Member Function Documentation

3.4.2.1 AppendRecord()

```
bool BlockedSequence::AppendRecord (
            const string & recordCsv)
```

Appends one packed record to the logical tail of the sequence.

Appends one packed record to the tail of the blocked sequence.

Parameters

	recordCsv	Length-indicated record payload to append.

Returns

True if the record is inserted into an existing or new block; otherwise false.

If the current tail block has space, the record is inserted there. If not, a new block is created, linked after the tail, and the record is inserted into the new block.

Parameters

	recordCsv	Length-indicated record payload to append.

Returns

True if appended successfully; otherwise false.

Creates the first block on demand. If the tail block is full, allocates and links a new tail block before insertion.

3.4.2.2 Delete()

```
bool BlockedSequence::Delete (
            const string & zipKey,
            string & logOutput)
```

Deletes a record by ZIP key, handling block merges/redistributions if needed.

Parameters

	zipKey	ZIP code to delete.
	logOutput	Reference to string where delete/merge/redistribution events are logged.

Returns

True if deletion succeeds; otherwise false.

Searches blocks for the key, removes it, and handles merge if block becomes underfull.

Parameters

	zipKey	ZIP code to delete.
	logOutput	Reference to string where delete/merge events are logged.

Returns

True if deletion succeeds; otherwise false.

3.4.2.3 DumpLogicalBlockOrder()

```
string BlockedSequence::DumpLogicalBlockOrder ()
```

Dumps active blocks in logical order (following next-RBN chain).

Returns

Formatted dump with list heads, links, and record keys per block.

3.4.2.4 DumpLogicalOrder()

```
string BlockedSequence::DumpLogicalOrder ()
```

Dumps the logical sequence of records from head to tail as a string.

Returns

A string representation of all records in logical order.

Traverses the active chain and concatenates the records from each block into a single string for display or debugging purposes.

3.4.2.5 DumpPhysicalBlockOrder()

```
string BlockedSequence::DumpPhysicalBlockOrder ()
```

Dumps active blocks in physical order (ascending RBN map order).

Returns

Formatted dump with list heads, links, and record keys per block.

3.4.2.6 GetAvailListSize()

```
int BlockedSequence::GetAvailListSize ()
```

Returns the number of blocks currently in the available list.

Returns

Count of blocks in the available list.

3.4.2.7 GetBlock()

```
Block * BlockedSequence::GetBlock (
            int rbn)
```

Looks up a block by relative block number.

Finds a block by its relative block number.

Parameters

	rbn	RBN key of the block to locate.

Returns

Pointer to the matching block, or nullptr if not found.

Parameters

	rbn	Block identifier to look up.

Returns

Pointer to the located block, or nullptr if the RBN is not present.

3.4.2.8 GetCount()

```
int BlockedSequence::GetCount ()
```

Returns the number of blocks currently tracked in memory.

Returns the number of blocks tracked by this sequence.

Returns

Count of blocks in the internal map.

Count of entries in the internal block map.

3.4.2.9 GetHeadRBN()

```
int BlockedSequence::GetHeadRBN ()
```

Returns the RBN of the head block.

Returns the head block RBN.

Returns

Head block RBN, or 0 when the sequence is empty.

RBN of the first block, or 0 if empty.

3.4.2.10 GetNextRBN()

```
int BlockedSequence::GetNextRBN ()
```

Returns the next RBN that will be assigned to a new block.

Returns the next available RBN for block creation.

Returns

Next available RBN value.

Next RBN counter value.

3.4.2.11 GetTailRBN()

```
int BlockedSequence::GetTailRBN ()
```

Returns the RBN of the tail block.

Returns the tail block RBN.

Returns

Tail block RBN, or 0 when the sequence is empty.

RBN of the last block, or 0 if empty.

3.4.2.12 Insert()

```
bool BlockedSequence::Insert (
            const string & recordCsv,
            string & logOutput)
```

Inserts a record into the appropriate block, handling block splits if needed.

Parameters

	recordCsv	Packed record to insert.
	logOutput	Reference to string where split/insert events are logged.

Returns

True if insert succeeds; otherwise false.

Searches existing blocks for the correct position, or creates a new block. If a block reaches capacity during insert, it is split and event is logged.

Parameters

	recordCsv	Packed record to insert.
	logOutput	Reference to string where split/insert events are logged.

Returns

True if insert succeeds; otherwise false.

3.4.2.13 IsEmpty()

```
bool BlockedSequence::IsEmpty ()
```

Indicates whether the sequence has any blocks.

Checks whether the sequence currently has no blocks.

Returns

True when no blocks are currently linked; otherwise false.

True when head RBN is 0; otherwise false.

3.4.2.14 LoadAllFromHead()

```
bool BlockedSequence::LoadAllFromHead ()
```

Reads every linked block from head to tail.

Reads all blocks from head to tail using next-RBN links.

Returns

True if all blocks are read successfully; otherwise false.

Traverses the active chain using each block's next-RBN pointer and calls Block::ReadBlock on each visited block.

Returns

True if every block read succeeds; otherwise false.

3.4.2.15 WriteAll()

```
bool BlockedSequence::WriteAll ()
```

Writes every linked block from head to tail.

Writes all blocks from head to tail using next-RBN links.

Returns

True if all blocks are written successfully; otherwise false.

Traverses the active chain using each block's next-RBN pointer and calls Block::WriteBlock on each visited block.

Returns

True if every block write succeeds; otherwise false.

The documentation for this class was generated from the following files:

- include/Project 3.0/BlockedSequence.h
- src/Project 3.0/BlockedSequence.cpp

3.5 CsvToLength Class Reference

```
#include <CsvToLength.h>
```

Public Member Functions

- CsvToLength (const string &filePath)
  Converts a source CSV file into a length-indicated output file.

3.5.1 Constructor & Destructor Documentation

3.5.1.1 CsvToLength()

```
CsvToLength::CsvToLength (
            const string & filePath)
```

Converts a source CSV file into a length-indicated output file.

Parameters

	filePath	Path to the source CSV file.

The constructor performs the conversion immediately by writing a generated header record and then writing each source row with a length prefix.

The documentation for this class was generated from the following files:

- include/CsvToLength.h
- src/CsvToLength.cpp

3.6 HeaderBuffer Class Reference

```
#include <HeaderBuffer.h>
```

Public Member Functions

- bool writeHeader (std::ostream &out, const HeaderRecord &header) const

  Writes the header record to the output stream.
- bool readHeader (std::istream &in, HeaderRecord &headerOut) const

  Reads and parses the header record from the input stream.

3.6.1 Member Function Documentation

3.6.1.1 readHeader()

```
bool HeaderBuffer::readHeader (
            std::istream & in,
            HeaderRecord & headerOut) const
```

Reads and parses the header record from the input stream.

Reads and parses the header from an input stream.

Parameters

	in	Input stream for the source length-indicated data file.
	headerOut	Output parameter populated with parsed header metadata.

Returns

true if the header was read and parsed successfully; otherwise false.

This method should be called before reading data records so record parsing logic can rely on header metadata.

Parameters

	in	Input stream for the source file.
	headerOut	Output metadata object populated from parsed header text.

Returns

true if the header was read and parsed; otherwise false.

3.6.1.2 writeHeader()

```
bool HeaderBuffer::writeHeader (
            std::ostream & out,
            const HeaderRecord & header) const
```

Writes the header record to the output stream.

Writes the serialized header to an output stream.

Parameters

	out	Output stream for the target length-indicated data file.
	header	Header metadata model to serialize and write.

Returns

true if the header was serialized and written successfully; otherwise false.

This method is expected to write the header as the first record in the data file using the project's canonical wire format.

Parameters

	out	Output stream for the destination file.
	header	Header metadata object to serialize and write.

Returns

true if the write operation succeeded; otherwise false.

The documentation for this class was generated from the following files:

- include/Project 3.0/HeaderBuffer.h
- src/Project 3.0/HeaderBuffer.cpp

3.7 HeaderRecord Class Reference

Metadata model for length-indicated and blocked-sequence file headers.

```
#include <HeaderRecord.h>
```

Public Member Functions

- HeaderRecord ()=default

  Constructs a default header record that can later be populated by parsing.
- HeaderRecord (const string &InFile)

  Constructs a HeaderRecord by inspecting an input CSV file.
- string GetHeader () const

  Returns the serialized header record string.
- bool WriteHeader (std::fstream &out, const HeaderRecord &header)

  Writes a serialized header record to a stream.
- bool ReadHeader (std::fstream &in, HeaderRecord &header)

  Reads and parses a serialized header record from a stream.
- void ConfigureBlockedSequenceHeader (const string &indexFileName, int recordCount, int blockCount, int blockSize, int minBlockCapacity, int availHeadRBN, int activeHeadRBN, bool staleFlag)

  Configures this header record for a blocked sequence set file.

Friends

- class HeaderBuffer

3.7.1 Detailed Description

Metadata model for length-indicated and blocked-sequence file headers.

3.7.2 Constructor & Destructor Documentation

3.7.2.1 HeaderRecord() [1/2]

```
HeaderRecord::HeaderRecord ()  [default]
```

Constructs a default header record that can later be populated by parsing.

3.7.2.2 HeaderRecord() [2/2]

```
HeaderRecord::HeaderRecord (
            const string & InFile)
```

Constructs a HeaderRecord by inspecting an input CSV file.

Builds the serialized header record from static schema values and CSV-derived record count.

Parameters

	InFile	Path to the source CSV file used to determine record count.
	InFile	Path to the source CSV file used to compute record count.

3.7.3 Member Function Documentation

3.7.3.1 ConfigureBlockedSequenceHeader()

```
void HeaderRecord::ConfigureBlockedSequenceHeader (
            const string & indexFileName,
            int recordCount,
            int blockCount,
            int blockSize,
            int minBlockCapacity,
            int availHeadRBN,
            int activeHeadRBN,
            bool staleFlag)
```

Configures this header record for a blocked sequence set file.

Configures blocked-sequence metadata fields and regenerates header text.

Parameters

indexFileName	Name/path of the simple index file.
recordCount	Number of records in active blocks.
blockCount	Number of active blocks.
blockSize	Block size in bytes.
minBlockCapacity	Minimum block capacity percentage (0-100).
availHeadRBN	RBN of avail-list head block.
activeHeadRBN	RBN of active-list head block.
staleFlag	True when the index/header is stale.
indexFileName	Index filename/path stored in the header metadata.
recordCount	Number of active records written into the sequence set.
blockCount	Number of active blocks in the sequence set.
blockSize	Block size in bytes.
minBlockCapacity	Minimum capacity percentage (0-100).
availHeadRBN	Avail-list head RBN.
activeHeadRBN	Active-list head RBN.
staleFlag	True when related metadata/index data is stale.

3.7.3.2 GetHeader()

```
string HeaderRecord::GetHeader () const
```

Returns the serialized header record string.

Returns the generated header line.

Returns

Header record text ready to be written to the output data file.

Serialized header record string.

3.7.3.3 ReadHeader()

```
bool HeaderRecord::ReadHeader (
            std::fstream & in,
            HeaderRecord & header)
```

Reads and parses a serialized header record from a stream.

Parameters

	in	Input stream containing a header line.
	header	Output header model populated from parsed data.

Returns

true when parse succeeds; otherwise false.

This legacy method remains for compatibility. Prefer HeaderBuffer::readHeader.

3.7.3.4 WriteHeader()

```
bool HeaderRecord::WriteHeader (
            std::fstream & out,
            const HeaderRecord & header)
```

Writes a serialized header record to a stream.

Parameters

	out	Output stream receiving the header line.
	header	Header model to serialize and write.

Returns

true when the write succeeds; otherwise false.

This legacy method remains for compatibility. Prefer HeaderBuffer::writeHeader.

3.7.4 Friends And Related Symbol Documentation

3.7.4.1 HeaderBuffer

```
friend class HeaderBuffer [friend]
```

The documentation for this class was generated from the following files:

- include/HeaderRecord.h
- src/HeaderRecord.cpp

3.8 Length_Indicated_ZipCodeBuffer Class Reference

Provides seqential access to zip code records from a data source.

```
#include <Length_Indicated_ZipCodeBuffer.h>
```

Public Member Functions

- Length_Indicated_ZipCodeBuffer (const std::string &filePath)

  Constructor that initializes the ZipCodeBuffer with the specified file path.
- bool readNext (ZipCodeRecord &recordOut)

  Reads the next CSV row.
- std::string packRecord (ZipCodeRecord &record) const

  Pack Record into length indicated file.
- ~Length_Indicated_ZipCodeBuffer ()

  Destructor that closes the file stream.

3.8.1 Detailed Description

Provides seqential access to zip code records from a data source.

Opens a CSV file Skips the header line Reads each line, parses it into a ZipCodeRecord, and stores it in a buffer for later retrieval. Returns one record at a time, allowing for efficient memory usage when processing large datasets.

3.8.2 Constructor & Destructor Documentation

3.8.2.1 Length_Indicated_ZipCodeBuffer()

```
Length_Indicated_ZipCodeBuffer::Length_Indicated_ZipCodeBuffer (
            const std::string & filePath)
```

Constructor that initializes the ZipCodeBuffer with the specified file path.

Constructor for Length_Indicated_ZipCodeBuffer that opens the specified length-indicated file.

Input file stream for reading zip code data

Parameters

	filePath	The path to the CSV file containing zip code data.
	filePath	The path to the length-indicated file containing zip code records.

Exceptions

	std::runtime_error	if the file cannot be opened.

Opens the file for reading length-indicated records. Each record is preceded by a size indicator that tells how many bytes the record contains.

3.8.2.2 ~Length_Indicated_ZipCodeBuffer()

```
Length_Indicated_ZipCodeBuffer::~Length_Indicated_ZipCodeBuffer ()
```

Destructor that closes the file stream.

Destructor for Length_Indicated_ZipCodeBuffer that closes the file stream if it is open.

Ensures proper cleanup of the file stream resource when the object is destroyed.

3.8.3 Member Function Documentation

3.8.3.1 packRecord()

```
std::string Length_Indicated_ZipCodeBuffer::packRecord (
            ZipCodeRecord & record) const
```

Pack Record into length indicated file.

Packs a ZipCodeRecord into a length-indicated format string.

Parameters

	record

Returns

Parameters

	record	The ZipCodeRecord to pack.

Returns

A string containing the packed record in "size,data" format where size is the byte count.

Converts the record fields into a comma-separated string, prefixes it with an ASCII size indicator, and returns the combined data suitable for adding to a block.

3.8.3.2 readNext()

```
bool Length_Indicated_ZipCodeBuffer::readNext (
            ZipCodeRecord & recordOut)
```

Reads the next CSV row.

Reads the next length-indicated record from the file and unpacks it into a ZipCodeRecord object.

Parameters

	record	Output parameter that will hold the next ZipCodeRecord read from the file.

Returns

true if a record was successfully read, false if end of file is reached.

Parameters

	recordOut	Output parameter that will hold the unpacked ZipCodeRecord.

Returns

true if a record was successfully read and unpacked, false if end of file is reached.

Reads a size indicator (integer) followed by the record data. The record is parsed into zip code, state, latitude, and longitude fields, then constructed into a ZipCodeRecord object.

The documentation for this class was generated from the following files:

- include/Length_Indicated_ZipCodeBuffer.h
- src/Length_Indicated_ZipCodeBuffer.cpp

3.9 PrimaryKeyIndex Class Reference

```
#include <PrimaryKeyIndex.h>
```

Public Member Functions

- PrimaryKeyIndex (const string &FilePath)

  Creates a PrimaryKeyIndex object rooted at a block directory or index path.
- void SetBlockDirectory (const string &blockDirectory)

  Sets the block directory used to resolve RBN -> block file path.
- bool BuildFromBlocks (const string &blockDirectory)

  Builds the in-memory index from the current block directory.
- ZipCodeRecord find (vector< string > Zips)

  Finds ZIP records for command-line style query arguments.
- bool WriteToFile (const string &path)

  Writes the in-memory simple index to a file.
- bool ReadFromFile (const string &path)

  Reads a serialized simple index file into memory.
- bool LoadFromIndexFile (const string &indexPath)

  Loads a simple index file into RAM and remembers its file path.
- string Dump () const

  Returns a readable dump of the current index contents.

3.9.1 Constructor & Destructor Documentation

3.9.1.1 PrimaryKeyIndex()

```
PrimaryKeyIndex::PrimaryKeyIndex (
            const string & FilePath)
```

Creates a PrimaryKeyIndex object rooted at a block directory or index path.

Constructs an index object rooted at a block directory.

Parameters

	FilePath	Path to a block directory (preferred) or index file.
	FilePath	Block directory path.

If the directory exists, attempts immediate in-memory build.

3.9.2 Member Function Documentation

3.9.2.1 BuildFromBlocks()

```
bool PrimaryKeyIndex::BuildFromBlocks (
            const string & blockDirectory)
```

Builds the in-memory index from the current block directory.

Builds an in-memory simple index from physical block files.

Returns

true if the index was built successfully; otherwise false.

Parameters

	blockDirectory	Directory containing block_<rbn>.blk files.

Returns

true when directory scan and index build complete; otherwise false.

Stores pairs <highest ZIP key in block, RBN> for active blocks.

3.9.2.2 Dump()

```
string PrimaryKeyIndex::Dump () const
```

Returns a readable dump of the current index contents.

Produces a readable dump of simple index entries.

Returns

Text listing highest-key / RBN pairs.

Newline-separated CSV lines in format <highestZipKey,RBN>.

3.9.2.3 find()

```
ZipCodeRecord PrimaryKeyIndex::find (
            vector< string > Zips)
```

Finds ZIP records for command-line style query arguments.

Searches requested ZIP keys using the simple block index.

Parameters

	Zips	ZIP query tokens (supports -Z<zip> form).

Returns

The first matching ZipCodeRecord, or an empty/default record when none are found.

Parameters

	Zips	CLI-style ZIP arguments (supports -Z<zip> or raw zip values).

Returns

First found ZipCodeRecord; default/empty record when none found.

Uses index to select candidate block, then scans that block only.

3.9.2.4 LoadFromIndexFile()

```
bool PrimaryKeyIndex::LoadFromIndexFile (
            const string & indexPath)
```

Loads a simple index file into RAM and remembers its file path.

Parameters

	indexPath	Path to simple index file.

Returns

true when index file was loaded successfully; otherwise false.

3.9.2.5 ReadFromFile()

```
bool PrimaryKeyIndex::ReadFromFile (
            const string & path)
```

Reads a serialized simple index file into memory.

Reads a previously written simple index file into memory.

Parameters

	path	Input path.

Returns

true if the index was successfully read; otherwise false.

Parameters

	path	Index file path.

Returns

true when parse succeeds; otherwise false.

3.9.2.6 SetBlockDirectory()

```
void PrimaryKeyIndex::SetBlockDirectory (
            const string & blockDirectory)
```

Sets the block directory used to resolve RBN -> block file path.

Parameters

	blockDirectory	Directory containing block_∗.blk files.

3.9.2.7 WriteToFile()

```
bool PrimaryKeyIndex::WriteToFile (
            const string & path)
```

Writes the in-memory simple index to a file.

Writes the current simple index map to disk.

Parameters

	path	Output path.

Returns

true if the index was successfully written; otherwise false.

Parameters

	path	Output index file path.

Returns

true when write succeeds; otherwise false.

The documentation for this class was generated from the following files:

- include/PrimaryKeyIndex.h
- src/PrimaryKeyIndex.cpp

3.10 Printer Class Reference

The Printer class is responsible for printing the results of the analysis in a formatted table. It provides a method to print the table of state extremes, which includes the easternmost,.

```
#include <Printer.h>
```

Public Member Functions

- void printTable (std::map< std::string, StateExtremes > results)

  Prints the table of state extremes, which includes the easternmost, westernmost, northernmost, and southernmost zip codes for each state.

3.10.1 Detailed Description

The Printer class is responsible for printing the results of the analysis in a formatted table. It provides a method to print the table of state extremes, which includes the easternmost,.

3.10.2 Member Function Documentation

3.10.2.1 printTable()

```
void Printer::printTable (
            std::map< std::string, StateExtremes > results)
```

Prints the table of state extremes, which includes the easternmost, westernmost, northernmost, and southernmost zip codes for each state.

Parameters

	results	A map of state extremes, where the key is the state ID and the value is a StateExtremes object containing the extreme

Output the column headers for the table of extremes.

Columns are fixed-width to provide a clean, aligned display. The values correspond to the state abbreviation and each directional extreme.

Lamba to convert a ZipCodeRecord into a printable string (May need to be tweaked depending on what ZipCodeRecord Contains)

The documentation for this class was generated from the following files:

- include/Printer.h
- src/printer.cpp

3.11 PrinterCSV Class Reference

The Printer class is responsible for printing the results of the analysis in a CSV file. It provides a method to print the table of state extremes, which includes the easternmost,.

```
#include <PrinterCSV.h>
```

Public Member Functions

- void printCSV (std::ostream &outFile, std::map< std::string, StateExtremes > results)

  Prints the table of state extremes, which includes the easternmost, westernmost, northernmost, and southernmost zip codes for each state.

3.11.1 Detailed Description

The Printer class is responsible for printing the results of the analysis in a CSV file. It provides a method to print the table of state extremes, which includes the easternmost,.

3.11.2 Member Function Documentation

3.11.2.1 printCSV()

```
void PrinterCSV::printCSV (
            std::ostream & outFile,
            std::map< std::string, StateExtremes > results)
```

Prints the table of state extremes, which includes the easternmost, westernmost, northernmost, and southernmost zip codes for each state.

Parameters

	results	A map of state extremes, where the key is the state ID and the value is a StateExtremes object containing the extreme

Output the column headers for the table of extremes.

Columns are fixed-width to provide a clean, aligned display. The values correspond to the state abbreviation and each directional extreme.

Lamba to convert a ZipCodeRecord into a printable string (May need to be tweaked depending on what ZipCodeRecord Contains)

The documentation for this class was generated from the following files:

- include/PrinterCSV.h
- src/printerCSV.cpp

3.12 RecordFormatter Class Reference

The RecordFormatter class provides methods for packing and unpacking ZipCodeRecord objects into a length-indicated format. It allows for efficient serialization and deserialization of ZipCodeRecord data, making it easier to store and transmit records in a compact format.

```
#include <RecordFormatter.h>
```

Static Public Member Functions

- static std::string pack (const ZipCodeRecord &record)
- static ZipCodeRecord unpack (const std::string &packedRecord)
- static std::string formatLabeled (const ZipCodeRecord &record)

  Formats a ZipCodeRecord into a labeled string for easy reading.

3.12.1 Detailed Description

The RecordFormatter class provides methods for packing and unpacking ZipCodeRecord objects into a length-indicated format. It allows for efficient serialization and deserialization of ZipCodeRecord data, making it easier to store and transmit records in a compact format.

3.12.2 Member Function Documentation

3.12.2.1 formatLabeled()

```
std::string RecordFormatter::formatLabeled (
            const ZipCodeRecord & record)  [static]
```

Formats a ZipCodeRecord into a labeled string for easy reading.

Example output: zipcode: 00501 | State: NY | Lat: 40.8154 | long: -73.0451

Parameters

	record	The record to format.

Returns

A single-line string with labeled fields.

3.12.2.2 pack()

```
std::string RecordFormatter::pack (
            const ZipCodeRecord & record)  [static]
```

3.12.2.3 unpack()

```
ZipCodeRecord RecordFormatter::unpack (
            const std::string & packedRecord)  [static]
```

The documentation for this class was generated from the following files:

- include/RecordFormatter.h
- src/RecordFormatter.cpp

3.13 StateExtremes Struct Reference

```
#include <StateExtremes.h>
```

Public Attributes

- bool initialized = false

  True once this state has at least one record.
- ZipCodeRecord easternmost

  ZipCodeRecord with least longitude for this state.
- ZipCodeRecord westernmost

  ZipCodeRecord with greatest longitude for this state.
- ZipCodeRecord northernmost

  ZipCodeRecord with greatest latitude for this state.
- ZipCodeRecord southernmost

  ZipCodeRecord with least latitude for this state.

3.13.1 Detailed Description

Option A : Data only struct - This just stores records and does not have independent behavior.

3.13.2 Member Data Documentation

3.13.2.1 easternmost

```
ZipCodeRecord StateExtremes::easternmost
```

ZipCodeRecord with least longitude for this state.

3.13.2.2 initialized

```
bool StateExtremes::initialized = false
```

True once this state has at least one record.

3.13.2.3 northernmost

```
ZipCodeRecord StateExtremes::northernmost
```

ZipCodeRecord with greatest latitude for this state.

3.13.2.4 southernmost

```
ZipCodeRecord StateExtremes::southernmost
```

ZipCodeRecord with least latitude for this state.

3.13.2.5 westernmost

```
ZipCodeRecord StateExtremes::westernmost
```

ZipCodeRecord with greatest longitude for this state.

The documentation for this struct was generated from the following file:

- include/StateExtremes.h

3.14 ZipCodeBuffer Class Reference

Provides seqential access to zip code records from a data source.

```
#include <ZipCodeBuffer.h>
```

Public Member Functions

- ZipCodeBuffer (const std::string &filePath)

  Constructor that initializes the ZipCodeBuffer with the specified file path.
- bool readNext (ZipCodeRecord &recordOut)

  Reads the next CSV row.
- ~ZipCodeBuffer ()

  Destructor that closes the file stream.

3.14.1 Detailed Description

Provides seqential access to zip code records from a data source.

Opens a CSV file Skips the header line Reads each line, parses it into a ZipCodeRecord, and stores it in a buffer for later retrieval. Returns one record at a time, allowing for efficient memory usage when processing large datasets.

3.14.2 Constructor & Destructor Documentation

3.14.2.1 ZipCodeBuffer()

```
ZipCodeBuffer::ZipCodeBuffer (
            const std::string & filePath)
```

Constructor that initializes the ZipCodeBuffer with the specified file path.

Constructor for ZipCodeBuffer that opens the specified CSV file and reads the header to determine column indices.

The index of the longitude column in the CSV file

Parameters

	filePath	The path to the CSV file containing zip code data.
	filePath	The path to the CSV file containing zip code data.

Exceptions

	std::runtime_error	if the file cannot be opened.

3.14.2.2 ~ZipCodeBuffer()

```
ZipCodeBuffer::~ZipCodeBuffer ()
```

Destructor that closes the file stream.

Destructor for ZipCodeBuffer that closes the file stream if it is open.

The destructor checks if the file stream is open and closes it to release any resources associated with the file. This ensures that the file is properly closed when the ZipCodeBuffer object goes out of scope or is destroyed.

3.14.3 Member Function Documentation

3.14.3.1 readNext()

```
bool ZipCodeBuffer::readNext (
            ZipCodeRecord & recordOut)
```

Reads the next CSV row.

Reads the next line from the CSV file and parses it into a ZipCodeRecord object.

Parameters

	record	Output parameter that will hold the next ZipCodeRecord read from the file.

Returns

true if a record was successfully read, false if end of file is reached.

Parameters

	recordOut	Output parameter that will hold the next ZipCodeRecord read from the file.

Returns

true if a record was successfully read, false if end of file is reached.

The method reads the next line from the CSV file, splits it into columns based on the previously determined indices, and constructs a ZipCodeRecord object with the parsed data. If the end of the file is reached, it returns false to indicate that there are no more records to read.

The documentation for this class was generated from the following files:

- include/ZipCodeBuffer.h
- src/ZipCodeBuffer.cpp

3.15 ZipCodeRecord Class Reference

```
#include <ZipCodeRecord.h>
```

Public Member Functions

- ZipCodeRecord ()

  Default constructor for ZipCodeRecord.
- ZipCodeRecord (std::string &zip, std::string &state, double &latitude, double &longitude)

  Constructor for ZipCodeRecord with parameters.
- const std::string & getState () const

  Getter for the state ID of the zip code record.
- const std::string & getZip () const

  Getter for the zip code of the zip code record.
- const double & getLatitude () const

  Getter for the latitude of the zip code record.
- const double & getLongitude () const

  Getter for the longitude of the zip code record.

3.15.1 Constructor & Destructor Documentation

3.15.1.1 ZipCodeRecord() [1/2]

```
ZipCodeRecord::ZipCodeRecord ()
```

Default constructor for ZipCodeRecord.

Default constructor for ZipCodeRecord that initializes all member variables to default values. The zip code and state are initialized to empty strings, while latitude and longitude are initialized to 0.0. This constructor allows for the creation of a ZipCodeRecord object without providing any initial data, which can be useful in situations where the record will be populated later or when creating placeholder records.

3.15.1.2 ZipCodeRecord() [2/2]

```
ZipCodeRecord::ZipCodeRecord (
            std::string & zip,
            std::string & state,
            double & latitude,
            double & longitude)
```

Constructor for ZipCodeRecord with parameters.

Constructor for ZipCodeRecord that initializes the member variables with the provided parameters.

Parameters

zip	
state	
latitude	
longitude	
zip	
state	
latitude	
longitude	The longitude of the city to be stored in the record. This constructor allows for the creation of a ZipCodeRecord objec

3.15.2 Member Function Documentation

3.15.2.1 getLatitude()

```
const double & ZipCodeRecord::getLatitude () const
```

Getter for the latitude of the zip code record.

Returns

A const reference to the latitude value

A const reference to the latitude value stored in the record. This method provides read-only access to the latitude of the record, allowing other parts of the program to retrieve this information without modifying it.

3.15.2.2 getLongitude()

```
const double & ZipCodeRecord::getLongitude () const
```

Getter for the longitude of the zip code record.

Returns

A const reference to the longitude value

A const reference to the longitude value stored in the record. This method provides read-only access to the longitude of the record, allowing other parts of the program to retrieve this information without modifying it.

3.15.2.3 getState()

```
const std::string & ZipCodeRecord::getState () const
```

Getter for the state ID of the zip code record.

Returns

A const reference to the state ID string

A const reference to the state ID string stored in the record. This method provides read-only access to the state ID of the zip code record, allowing other parts of the program to retrieve this information without modifying it.

3.15.2.4 getZip()

```
const std::string & ZipCodeRecord::getZip () const
```

Getter for the zip code of the zip code record.

Returns

A const reference to the zip code string

A const reference to the zip code string stored in the record. This method provides read-only access to the zip code of the record, allowing other parts of the program to retrieve this information without modifying it.

The documentation for this class was generated from the following files:

- include/ZipCodeRecord.h
- src/ZipCodeRecord.cpp

Chapter 4

File Documentation

4.1 include/analyzer.h File Reference

Header file for the Analyzer class, which provides methods for analyzing data sets.

```
#include <map>
#include <string>
#include "ZipCodeRecord.h"
#include "StateExtremes.h"
```

Classes

- class Analyzer

4.1.1 Detailed Description

Header file for the Analyzer class, which provides methods for analyzing data sets.

Author

Robert Shores & Zach Houck

Date

2026-02-10

4.2 analyzer.h

Go to the documentation of this file.

```
00001
00007
00008  #ifndef ANALYZER_H
00009  #define ANALYZER_H
00010
00011
00012
00013  #include <map>
00014  #include <string>
00015  #include "ZipCodeRecord.h"
00016  #include "StateExtremes.h"
00017
00018  class Analyzer
00019     {
00020
00021         private:
00025             std::map<std::string, StateExtremes> results;
00026
00027
00028
00029
00030         public:
00037
00038          void consume(const ZipCodeRecord& record);
00039
00040
00049
00050          const std::map<std::string, StateExtremes>& getResults() const;
00051
00052
00053
00054     };
00055
00056 #endif // ANALYZER_H
```

4.3 include/CsvToLength.h File Reference

Declares CsvToLength, a utility class that converts a CSV file into a length-indicated output file.

```
#include <iostream>
#include <fstream>
#include <filesystem>
#include <string>
#include <memory>
```

Classes

- class CsvToLength

4.3.1 Detailed Description

Declares CsvToLength, a utility class that converts a CSV file into a length-indicated output file.

Author

Zach Houck

Date

2026-02-10

4.4 CsvToLength.h

Go to the documentation of this file.

```
00001
00007
00008 #ifndef  CSVTOLENGTH_H
00009 #define  CSVTOLENGTH_H
00010
00011 #include <iostream>
00012 #include <fstream>
00013 #include <filesystem>
00014 #include <string>
00015 #include <memory>
00016
00017 using namespace std;
00018
00019 class CsvToLength
00020 {
00021     private:
00022
00023     public:
00030     CsvToLength(const string& filePath);
00031
00032 };
00033
00034 #endif
```

4.5 include/HeaderRecord.h File Reference

Declares the HeaderRecord class used to build metadata for a length-indicated data file.

```
#include <iostream>
#include <fstream>
#include <filesystem>
#include <string>
#include <vector>
```

Classes

- class HeaderRecord

  Metadata model for length-indicated and blocked-sequence file headers.

4.5.1 Detailed Description

Declares the HeaderRecord class used to build metadata for a length-indicated data file.

Author

Zach Houck

Date

2026-02-11

4.6 HeaderRecord.h

Go to the documentation of this file.

```
00001
00007
00008 #ifndef HEADERRECORD_H
00009 #define HEADERRECORD_H
00010
00011 #include <iostream>
00012 #include <fstream>
00013 #include <filesystem>
00014 #include <string>
00015 #include <vector>
00016
00017 using namespace std;
00018
00022 class HeaderRecord
00023 {
00024     friend class HeaderBuffer;
00025
00026     private:
00027
00028
00029
00030
00031
00032
00033
00034
00035         int HeaderSizeBytes = 0;
00036         string FileStructureType = "CSV";
00037         string Version = "1.0";
00038         bool SizeFormatType = 0;
00039         bool SizeInclusion = true;
00040         string PrimaryKeyIndex = "Key_index.txt";
00041         int RecordCount = 0;
00042         int FieldCount = 6;
00043         vector<string> Fields = {"ZipCode", "City", "State", "County", "Lat", "Long"};
00044         vector<string> FieldTypes = {"INT", "STRING", "STRING", "STRING", "FLOAT", "FLOAT"};
00045         int PrimaryKey = 1;
00046
00047         string Header = "";
00048
00050         void RebuildHeader();
00051
00052
00053
00054
00055         int BlockSize = 512;
00056         int MinBlockCapacity = 50;
00057         int BlockCount = 0;
00058         int availListHeadRBN = 0;
00059         int ActiveListHeadRBN = 0;
00060         bool StaleFlag = false;
00061
00062
00063
00064     public:
00068     HeaderRecord() = default;
00069
00074     HeaderRecord(const string& InFile);
00075
00080     string GetHeader() const;
00081
00082
00083
00084     // Project 3 //
00085     //
00086
00094     bool WriteHeader(std::fstream& out, const HeaderRecord& header);
00095
00103     bool ReadHeader(std::fstream& in, HeaderRecord& header);
00104
00116         void ConfigureBlockedSequenceHeader(
00117             const string& indexFileName,
00118             int recordCount,
00119             int blockCount,
00120             int blockSize,
00121             int minBlockCapacity,
00122             int availHeadRBN,
00123             int activeHeadRBN,
00124             bool staleFlag);
00125
00126 };
00127
```

```
#endif
```

4.7 include/Length_Indicated_ZipCodeBuffer.h File Reference

Declares the LengthIndicatedZipCodeBuffer class, which reads and writes length indicated Zip Code records.

```
#include <iostream>
#include <fstream>
#include <string>
#include <memory>
#include "ZipCodeRecord.h"
```

Classes

- class Length_Indicated_ZipCodeBuffer

  Provides seqential access to zip code records from a data source.

4.7.1 Detailed Description

Declares the LengthIndicatedZipCodeBuffer class, which reads and writes length indicated Zip Code records.

Author

Ken Tabata & Gabriel K Coffie Boafo & Robert Shores

Date

2026-02-10

4.8 Length_Indicated_ZipCodeBuffer.h

Go to the documentation of this file.

```
#ifndef  LENGTH_INDICATED_ZIPCODEBUFFER_H
#define  LENGTH_INDICATED_ZIPCODEBUFFER_H

#include <iostream>
#include <fstream>
#include <string>
#include <memory>
#include "ZipCodeRecord.h"


class Length_Indicated_ZipCodeBuffer{
   private:
         std::ifstream file;


   public:
       Length_Indicated_ZipCodeBuffer(const std::string& filePath);


       bool readNext(ZipCodeRecord& recordOut);



       std::string packRecord(ZipCodeRecord& record) const;



       ~Length_Indicated_ZipCodeBuffer();
 };
// #include "ZipCodeBuffer.cpp" // This is not needed since we will compile the .cpp file separately, and including it here can lead to multiple definition errors.
#endif
```

4.9 include/PrimaryKeyIndex.h File Reference

Declares PrimaryKeyIndex, a primary key index that can be used to display the Zip Code data for all Zip Codes listed on the command line.

```
#include <iostream>
#include <fstream>
#include <filesystem>
#include <string>
#include <vector>
#include <memory>
#include <map>
#include "ZipCodeRecord.h"
```

Classes

- class PrimaryKeyIndex

4.9.1 Detailed Description

Declares PrimaryKeyIndex, a primary key index that can be used to display the Zip Code data for all Zip Codes listed on the command line.

Author

Zach Houck

Date

2026-03-14

4.10 PrimaryKeyIndex.h

Go to the documentation of this file.

```
00001
00007
00008 #ifndef  PRIMARYKEYINDEX_H
00009 #define  PRIMARYKEYINDEX_H
00010
00011 #include <iostream>
00012 #include <fstream>
00013 #include <filesystem>
00014 #include <string>
00015 #include <vector>
00016 #include <memory>
00017 #include <map>
00018
00019 #include "ZipCodeRecord.h"
00020
00021 using namespace std;
00022
00023 class PrimaryKeyIndex
00024 {
00025     private:
00026
00030         map<string, int> Index;
00031
00033         string BlockDirectory = "data/blocks";
```

```
        string IndexFilePath = "";

        static string ExtractZipKey(const string& packedRecord);

        string GetBlockFilePath(int rbn) const;

    public:

        PrimaryKeyIndex(const string& FilePath);

        void SetBlockDirectory(const string& blockDirectory);

        bool BuildFromBlocks(const string& blockDirectory);

        ZipCodeRecord find(vector<string> Zips);

        bool WriteToFile(const string& path);

        bool ReadFromFile(const string& path);

        bool LoadFromIndexFile(const string& indexPath);

        string Dump() const;
};

#endif
```

4.11 include/Printer.h File Reference

Header file for the Printer class, which provides methods for printing the results of the analysis.

```
#include "StateExtremes.h"
#include <map>
```

Classes

- class Printer

  The Printer class is responsible for printing the results of the analysis in a formatted table. It provides a method to print the table of state extremes, which includes the easternmost,.

4.11.1 Detailed Description

Header file for the Printer class, which provides methods for printing the results of the analysis.

Author

Robert Shores

Date

2026-02-10

4.12 Printer.h

Go to the documentation of this file.

```
00001
00007
00008 #ifndef PRINTER_H
00009 #define PRINTER_H
00010
00011 #include "StateExtremes.h"
00012 #include <map>
00013
00014
00019 class Printer
00020    {
00021     public:
00022
00027     void printTable(std::map<std::string, StateExtremes> results);
00028
00029    };
00030
00031 #endif
```

4.13 include/PrinterCSV.h File Reference

Header file for the PrinterCSV class, which provides methods for printing the results of the analysis into a CSV file.

```
#include "StateExtremes.h"
#include <iostream>
#include <fstream>
#include <map>
```

Classes

- class PrinterCSV

  The Printer class is responsible for printing the results of the analysis in a CSV file. It provides a method to print the table of state extremes, which includes the easternmost,.

4.13.1 Detailed Description

Header file for the PrinterCSV class, which provides methods for printing the results of the analysis into a CSV file.

Author

Robert Shores & Zach Houck

Date

2026-02-10

4.14 PrinterCSV.h

Go to the documentation of this file.

```
00001
00007
00008 #ifndef PRINTERCSV_H
00009 #define PRINTERCSV_H
00010
00011 #include "StateExtremes.h"
00012 #include <iostream>
00013 #include <fstream>
00014 #include <map>
00015
00016
00021 class PrinterCSV
00022    {
00023     public:
00024
00029     void printCSV(std::ostream& outFile, std::map<std::string, StateExtremes> results);
00030
00031    };
00032
00033 #endif
```

4.15 include/Project 3.0/Block.h File Reference

Declares the Block container used to store packed records within a fixed byte-capacity unit.

```
#include <iostream>
#include <string>
#include <vector>
```

Classes

- class Block

4.15.1 Detailed Description

Declares the Block container used to store packed records within a fixed byte-capacity unit.

Author

Zach Houck

Date

2026-03-26

Each block tracks metadata (record count, RBN links, byte usage) and stores length-indicated record payloads.

4.16 Block.h

Go to the documentation of this file.

```
00001
00011
00012 #ifndef BLOCK_H
00013 #define BLOCK_H
00014
00015 #include <iostream>
00016 #include <string>
00017 #include <vector>
00018
00019 using namespace std;
00020
00021 class Block
00022 {
00023     private:
00024
00026         int RecordCount = 0;
00028         int RBN = 0;
00030         int PrevRBN = 0;
00032         int NextRBN = 0;
00034         int ByteSize = 0;
00036         int ByteMaxSize = 512; //512 is default
00038         int ByteMinSize = 256; //256 is default
00040         int HeaderSize = 0;
00041
00043         string BlockHeader;
00045         vector<string> Records;
00046
00048         void UpdateHeader();
00049
00050     public:
00051
00057         Block(int prevrbn, int rbn);
00058
00065         bool AddRecord(const string& Record);
00066
00072         bool RemoveRecordAt(size_t recordIndex);
00073
00078         bool ReadBlock();
00079
00084         bool WriteBlock();
00085
00090         int GetRecordCount();
00091
00096         int GetRBN();
00097
00102         int GetPrevRBN();
00103
00108         int GetNextRBN();
00109
00114         int GetByteSize();
00115
00120         int GetByteMaxSize();
00121
00126         int GetByteMinSize();
00127
00132         vector<string>& GetRecords();
00133
00138         void SetNextRBN(int nextRbn);
00139
00144         void SetPrevRBN(int prevRbn);
00145
00146 };
00147
00148 #endif
```

4.17 include/Project 3.0/BlockBuffer.h File Reference

Declares BlockBuffer utilities for packing/unpacking block and record data.

```
#include <iostream>
#include <fstream>
#include <string>
#include <memory>
```

```
#include <cstring>
#include "Block.h"
#include "ZipCodeRecord.h"
```

Classes

- class BlockBuffer

  Buffer helper for reading/writing packed fields and unpacking records from blocks.

4.17.1 Detailed Description

Declares BlockBuffer utilities for packing/unpacking block and record data.

Author

Robert Shores

Date

2026-03-31

4.18 BlockBuffer.h

Go to the documentation of this file.

```
00001
00007
00008
00009
00010 #ifndef BLOCKBUFFER_H
00011 #define BLOCKBUFFER_H
00012
00013 #include <iostream>
00014 #include <fstream>
00015 #include <string>
00016 #include <memory>
00017 #include <cstring>
00018 #include "Block.h"
00019 #include "ZipCodeRecord.h"
00020
00024 class BlockBuffer {
00025     private:
00026         std::ifstream file;
00027         std::string csvPath;
00028         bool BufferRead;
00029
00030     public:
00031
00036         BlockBuffer(int maxBytes = 512);
00037
00041         void Clear();
00042
00048         int Read(istream& stream);
00049
00055         int Write(ostream& stream) const;
00056
00063         int Pack(const char*, short size = -1);
00064
00070         int Unpack(char*);
00071
00076         void Print(ostream&) const;
00077
00083         int Init(int maxBytes = 512);
00084
00092         bool UnpackRecordFromBlock(Block& block, int recordIndex, std::string& recordBufferOut) const;
```

```
        bool UnpackFieldsToRecord(const std::string& recordBuffer, ZipCodeRecord& recordOut) const;

        bool UnpackRecordFromBlock(Block& block, int recordIndex, ZipCodeRecord& recordOut) const;

    private:

        char* Buffer;
        int BufferSize;
        int MaxBytes;
        int NextByte;
};

#endif
```

4.19 include/Project 3.0/BlockedSequence.h File Reference

Declares the BlockedSequence container used to manage linked blocks by RBN.

```
#include <map>
#include <queue>
#include <string>
#include "Block.h"
```

Classes

- class BlockedSequence

4.19.1 Detailed Description

Declares the BlockedSequence container used to manage linked blocks by RBN.

Author

Zach Houck

Date

2026-03-26

The sequence stores blocks in memory and maintains logical ordering through each block's previous and next relative block number (RBN).

4.20 BlockedSequence.h

Go to the documentation of this file.

```
00001
00010
00011 #ifndef BLOCKEDSEQUENCE_H
00012 #define BLOCKEDSEQUENCE_H
00013
00014 #include <map>
00015 #include <queue>
00016 #include <string>
00017
00018 #include "Block.h"
00019
00020 using namespace std;
00021
00022 class BlockedSequence
00023 {
00024     private:
00026         map<int, Block> blocks;
00028         int HeadRBN = 0;
00030         int TailRBN = 0;
00032         int NextRBN = 1;
00034         queue<int> availList;
00035
00036     public:
00037
00044         BlockedSequence(int startRBN = 1);
00045
00050         bool IsEmpty();
00051
00056         int GetHeadRBN();
00057
00062         int GetTailRBN();
00063
00068         int GetNextRBN();
00069
00074         int GetCount();
00075
00085         bool AppendRecord(const string& recordCsv);
00086
00093         bool WriteAll();
00094
00101         bool LoadAllFromHead();
00102
00108         Block* GetBlock(int rbn);
00109
00110
00117         string DumpLogicalOrder();
00118
00123         string DumpPhysicalBlockOrder();
00124
00129         string DumpLogicalBlockOrder();
00130
00139         bool Insert(const string& recordCsv, string& logOutput);
00140
00148         bool Delete(const string& zipKey, string& logOutput);
00149
00154         int GetAvailListSize();
00155 };
00156
00157 #endif
```

4.21 include/Project 3.0/HeaderBuffer.h File Reference

Declares the HeaderBuffer class for reading and writing header records.

```
#include <istream>
#include <ostream>
#include <string>
#include "HeaderRecord.h"
```

Classes

- class HeaderBuffer

4.21.1 Detailed Description

Declares the HeaderBuffer class for reading and writing header records.

Author

Robert Shores

Date

2026-02-10

HeaderBuffer serializes and deserializes the metadata record stored at the beginning of a length-indicated data file.

4.22 HeaderBuffer.h

Go to the documentation of this file.

```
00001
00009
00010 #ifndef  HEADBUFFER_H
00011 #define  HEADBUFFER_H
00012
00013 #include <istream>
00014 #include <ostream>
00015 #include <string>
00016 #include "HeaderRecord.h"
00017
00018 class HeaderBuffer {
00019     public:
00020
00029         bool writeHeader(std::ostream& out, const HeaderRecord& header) const;
00030
00031
00032
00041         bool readHeader(std::istream& in, HeaderRecord& headerOut) const;
00042
00043     private:
00044
00050         std::string packHeader(const HeaderRecord& header) const;
00051
00058         bool unpackHeader(const std::string& payload, HeaderRecord& headerOut) const;
00059
00060
00061
00062 };
00063
00064 #endif
```

4.23 include/RecordFormatter.h File Reference

Declares the RecordFormatter class, which provides methods for packing and unpacking ZipCodeRecord objects into a length-indicated format.

```
#include "ZipCodeRecord.h"
#include <string>
#include <sstream>
```

Classes

- class RecordFormatter

  The RecordFormatter class provides methods for packing and unpacking ZipCodeRecord objects into a length-indicated format. It allows for efficient serialization and deserialization of ZipCodeRecord data, making it easier to store and transmit records in a compact format.

4.23.1 Detailed Description

Declares the RecordFormatter class, which provides methods for packing and unpacking ZipCodeRecord objects into a length-indicated format.

Author

Gabriel Kofi Coffie Boafo & Ken Tabata

Date

2026-02-14

4.24 RecordFormatter.h

Go to the documentation of this file.

```
00001
00007
00008     #ifndef RECORDFORMATTER_H
00009     #define RECORDFORMATTER_H
00010
00011    #include "ZipCodeRecord.h"
00012    #include <string>
00013    #include <sstream>
00014
00019     class RecordFormatter {
00020     public:
00021
00022       //   Packs a ZipCodeRecord into a length-indicated string format.
00023
00024         static std::string pack(const ZipCodeRecord& record);
00025
00026          // Unpacks a length-indicated string into a ZipCodeRecord object.
00027
00028         static ZipCodeRecord unpack(const std::string& packedRecord);
00029
00039         static std::string formatLabeled(const ZipCodeRecord& record);
00040     };
00041
00042     #endif // RECORDFORMATTER_H
```

4.25 include/StateExtremes.h File Reference

Header file for the StateExtremes class, which provides methods for analyzing data sets.

```
#include "ZipCodeRecord.h"
```

Classes

- struct StateExtremes

4.25.1 Detailed Description

Header file for the StateExtremes class, which provides methods for analyzing data sets.

Author

Robert Shores & Zach Houck

Date

2026-02-12

4.26 StateExtremes.h

Go to the documentation of this file.

```
00001
00007
00008 #ifndef STATEEXTREMES_H
00009 #define STATEEXTREMES_H
00010
00011
00012 #include "ZipCodeRecord.h"
00013
00018
00019 struct StateExtremes
00020 {
00022     bool initialized = false;
00023
00025     ZipCodeRecord easternmost;
00026
00028     ZipCodeRecord westernmost;
00029
00031     ZipCodeRecord northernmost;
00032
00034     ZipCodeRecord southernmost;
00035 };
00036
00037
00038
00043 // class StateExtremes
00044
00045 //    {
00046 //       private:
00047
00048 //          /**
00049 //          * @brief initialized is false until the first record is consumed, then it is set to true
      and the min and max values are updated accordingly
00050 //          */
00051 //          bool initialized = false;
00052
00053 //          /**
00054 //          * @brief Stores a ZipCodeRecord for all of the directions extremes
00055 //          */
00056 //          ZipCodeRecord easternmost;
00057 //          ZipCodeRecord westernmost;
00058 //          ZipCodeRecord northernmost;
00059 //          ZipCodeRecord southernmost;
00060
00061 //       public:
00062
00063 //       /**
00064 //       * @brief Default Constructor
00065 //       * @post creates an uninitialized StateExtremes object
00066 //       */
00067 //       StateExtremes();
00068
00069 //       /**
00070 //       * @brief Constructor
00071 //       * @param FirstZip the ZipCodeRecord used to create a StateExtremes object
00072 //       * @post creates an initialized StateExtremes object
00073 //       */
00074 //       StateExtremes(ZipCodeRecord FirstZip);
00075
00076 //       /**
```

```
//       * @brief Get functions
//       */
//       bool isInitialized();
//       ZipCodeRecord getEastern();
//       ZipCodeRecord getWestern();
//       ZipCodeRecord getNorthern();
//       ZipCodeRecord getSouthern();

//       /**
//       * @brief Set functions
//       */
//       void setInitialized(bool init);
//       void setEastern(ZipCodeRecord zip);
//       void setWestern(ZipCodeRecord zip);
//       void setNorthern(ZipCodeRecord zip);
//       void setSouthern(ZipCodeRecord zip);
//    };



#endif // STATE_EXTREMES_H
```

4.27 include/ZipCodeBuffer.h File Reference

This declares the ZipCodeBuffer class, which is responsible for buffering and managing zip code data for efficient access and processing.

```
#include <iostream>
#include <fstream>
#include <string>
#include <memory>
#include "ZipCodeRecord.h"
```

Classes

- class ZipCodeBuffer

  Provides seqential access to zip code records from a data source.

4.27.1 Detailed Description

This declares the ZipCodeBuffer class, which is responsible for buffering and managing zip code data for efficient access and processing.

Author

Ken Tabata & Gabriel K Coffie Boafo

Date

2026-02-10

4.28 ZipCodeBuffer.h

Go to the documentation of this file.

```
00001
00007
00008 #ifndef  ZIPCODEBUFFER_H
00009 #define  ZIPCODEBUFFER_H
00010
00011 #include <iostream>
00012 #include <fstream>
00013 #include <string>
00014 #include <memory>
00015 #include "ZipCodeRecord.h"
00016
00025
00026 class ZipCodeBuffer{
00027    private:
00028          std::ifstream file;
00029          std::string csvPath;
00030          bool headerRead;
00031          int zipIndex;
00032          int stateIndex;
00033          int latIndex;
00034          int lonIndex;
00035
00036    public:
00041        ZipCodeBuffer(const std::string& filePath);
00047        bool readNext(ZipCodeRecord& recordOut);
00048
00052        ~ZipCodeBuffer();
00053 };
00054 // #include "ZipCodeBuffer.cpp" // This is not needed since we will compile the .cpp file separately,
      and including it here can lead to multiple definition errors.
00055 #endif
```

4.29 include/ZipCodeRecord.h File Reference

Header file for the ZipCodeRecord class, which provides a data structure for storing zip code information.

```
#include <string>
```

Classes

- class ZipCodeRecord

4.29.1 Detailed Description

Header file for the ZipCodeRecord class, which provides a data structure for storing zip code information.

Author

Ken Tabata & Gabriel K.Coffie Boafo

Date

2026-02-12

4.30 ZipCodeRecord.h

Go to the documentation of this file.

```
#include <string>

#ifndef ZIPCODERECORD_H
#define ZIPCODERECORD_H

class ZipCodeRecord
   {
       public:
           ZipCodeRecord();

           ZipCodeRecord(std::string& zip, std::string& state, double& latitude, double& longitude);

           const std::string& getState() const;

           const std::string& getZip() const;

           const double& getLatitude() const;

           const double& getLongitude() const;

       private:

           std::string zip;
           std::string state;
           double latitude;
           double longitude;
   };
#endif
```

4.31 src/analyzer.cpp File Reference

Implementation of the Analyzer class for data processing.

```
#include "../include/analyzer.h"
#include "../include/StateExtremes.h"
```

4.31.1 Detailed Description

Implementation of the Analyzer class for data processing.

Author

Robert Shores

Date

2026-02-12

4.32 src/CsvToLength.cpp File Reference

Implements CSV-to-length-indicated conversion logic.

```
#include <iostream>
#include <iomanip>
#include <fstream>
#include <sstream>
#include <filesystem>
#include <string>
#include <memory>
#include "../include/CsvToLength.h"
#include "../include/HeaderRecord.h"
```

Macros

- #define CSVTOLENGTH_CPP

4.32.1 Detailed Description

Implements CSV-to-length-indicated conversion logic.

Author

Zach Houck

Date

2026-02-10

4.32.2 Macro Definition Documentation

4.32.2.1 CSVTOLENGTH_CPP

```
#define CSVTOLENGTH_CPP
```

4.33 src/driver_csv_to_length_indicated.cpp File Reference

driver for csv to length indicated file

```
#include <iostream>
#include <fstream>
#include <string>
#include "../include/CsvToLength.h"
```

Functions

- int main ()

4.33.1 Detailed Description

driver for csv to length indicated file

Author

Ken, Zach, Gabriel, Robert

Date

2026-02-10

4.33.2 Function Documentation

4.33.2.1 main()

```
int main ()
```

4.34 src/HeaderRecord.cpp File Reference

Implements HeaderRecord, which builds a serialized header record for the length-indicated file format.

```
#include <iostream>
#include <fstream>
#include <filesystem>
#include <string>
#include <sstream>
#include "../include/HeaderRecord.h"
```

Macros

- #define HEADERRECORD_CPP

Functions

- string operator+ (const string &s, int value)
- string operator+ (int value, const string &s)

4.34.1 Detailed Description

Implements HeaderRecord, which builds a serialized header record for the length-indicated file format.

Author

Zach Houck

Date

2026-02-11

4.34.2 Macro Definition Documentation

4.34.2.1 HEADERRECORD_CPP

```
#define HEADERRECORD_CPP
```

4.34.3 Function Documentation

4.34.3.1 operator+() [1/2]

```
string operator+ (
            const string & s,
            int value)
```

4.34.3.2 operator+() [2/2]

```
string operator+ (
            int value,
            const string & s)
```

4.35 src/Length_Indicated_ZipCodeBuffer.cpp File Reference

Implementation file for the Length_Indicated_ZipCodeBuffer class, which provides methods for reading and writing length-indicated Zip Code records.

```
#include "../include/Length_Indicated_ZipCodeBuffer.h"
#include <stdexcept>
#include <sstream>
#include <iomanip>
#include <cstring>
#include <cstdint>
```

4.35.1 Detailed Description

Implementation file for the Length_Indicated_ZipCodeBuffer class, which provides methods for reading and writing length-indicated Zip Code records.

Author

Ken Tabata & Gabriel K Coffie Boafo & Robert Shoes

Date

2026-02-11

4.36 src/main.cpp File Reference

Production CLI entry point for Project 3.0 blocked-sequence generation.

```
#include <filesystem>
#include <fstream>
#include <iostream>
#include <sstream>
#include <stdexcept>
#include <string>
#include "Length_Indicated_ZipCodeBuffer.h"
#include "Project 3.0/BlockedSequence.h"
#include "Project 3.0/HeaderBuffer.h"
#include "HeaderRecord.h"
#include "PrimaryKeyIndex.h"
```

Functions

- int main (int argc, char ∗argv[ ])

  Main entry point for blocked-sequence generation and optional mutations.

4.36.1 Detailed Description

Production CLI entry point for Project 3.0 blocked-sequence generation.

4.36.2 Function Documentation

4.36.2.1 main()

```
int main (
            int argc,
            char * argv[ ])
```

Main entry point for blocked-sequence generation and optional mutations.

Parameters

	argc	Argument count.
	argv	Argument vector.

Returns

Process exit code.

4.37 src/PrimaryKeyIndex.cpp File Reference

Implements a primary key index that can be used to display the Zip Code data for all Zip Codes listed on the command line.

```
#include <iostream>
#include <fstream>
#include <filesystem>
#include <sstream>
#include <string>
#include <vector>
#include <memory>
#include <map>
#include <algorithm>
#include "PrimaryKeyIndex.h"
#include "ZipCodeRecord.h"
#include "Project 3.0/BlockBuffer.h"
```

Macros

- #define PRIMARYKEYINDEX_CPP

4.37.1 Detailed Description

Implements a primary key index that can be used to display the Zip Code data for all Zip Codes listed on the command line.

Author

Zach Houck

Date

2026-03-14

4.37.2 Macro Definition Documentation

4.37.2.1 PRIMARYKEYINDEX_CPP

```
#define PRIMARYKEYINDEX_CPP
```

4.38 src/printer.cpp File Reference

Implementation file for the Printer class, which provides methods for printing the results of the analysis.

```
#include "../include/Printer.h"
#include "../include/analyzer.h"
#include <iostream>
#include <iomanip>
#include <sstream>
```

Macros

- #define PRINTER_CPP

4.38.1 Detailed Description

Implementation file for the Printer class, which provides methods for printing the results of the analysis.

Author

Robert Shores & Dade Kutzera

Date

2026-02-10

4.38.2 Macro Definition Documentation

4.38.2.1 PRINTER_CPP

```
#define PRINTER_CPP
```

4.39 src/printerCSV.cpp File Reference

Implementation file for the PrinterCSV class, which provides methods for printing the results of the analysis.

```
#include "../include/PrinterCSV.h"
#include "../include/analyzer.h"
#include <iostream>
#include <fstream>
#include <iomanip>
#include <sstream>
```

Macros

- #define PRINTERCSV_CPP

4.39.1 Detailed Description

Implementation file for the PrinterCSV class, which provides methods for printing the results of the analysis.

Author

Robert Shores, Dade Kutzera, Zach Houck

Date

2026-02-10

4.39.2 Macro Definition Documentation

4.39.2.1 PRINTERCSV_CPP

```
#define PRINTERCSV_CPP
```

4.40 src/Project 3.0/Block.cpp File Reference

Implements Block behavior for record storage, metadata updates, and capacity checks.

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <filesystem>
#include "Block.h"
```

Macros

- #define BLOCK_CPP

4.40.1 Detailed Description

Implements Block behavior for record storage, metadata updates, and capacity checks.

Author

Zach Houck

Date

2026-03-26

4.40.2 Macro Definition Documentation

4.40.2.1 BLOCK_CPP

```
#define BLOCK_CPP
```

4.41 src/Project 3.0/BlockBuffer.cpp File Reference

Implementation file for the BlockBuffer class which unpacks a record from a block into a record buffer. Reads/writes whole fixed-size blocks (default 512 bytes).

```
#include "Project 3.0/BlockBuffer.h"
#include <string.h>
#include <sstream>
```

4.41.1 Detailed Description

Implementation file for the BlockBuffer class which unpacks a record from a block into a record buffer. Reads/writes whole fixed-size blocks (default 512 bytes).

Author

Robert Shores

Date

2026-03-31

Knows block meta data layout: record count, prev RBN, next RBN

4.42 src/Project 3.0/BlockedSequence.cpp File Reference

Implements BlockedSequence operations for a doubly-linked block chain.

```
#include <map>
#include <string>
#include <sstream>
#include <vector>
#include <filesystem>
#include "BlockedSequence.h"
#include "Block.h"
```

Macros

- #define BLOCKEDSEQUENCE_CPP

4.42.1 Detailed Description

Implements BlockedSequence operations for a doubly-linked block chain.

Author

Zach Houck

Date

2026-03-26

This module manages in-memory block allocation, append behavior, and head-to-tail traversal for read/write operations.

4.42.2 Macro Definition Documentation

4.42.2.1 BLOCKEDSEQUENCE_CPP

```
#define BLOCKEDSEQUENCE_CPP
```

4.43 src/Project 3.0/HeaderBuffer.cpp File Reference

Implements HeaderBuffer methods for header-record serialization.

```
#include "Project 3.0/HeaderBuffer.h"
#include <sstream>
#include <vector>
```

Macros

- #define HEADBUFFER_CPP

4.43.1 Detailed Description

Implements HeaderBuffer methods for header-record serialization.

Author

Robert Shores

Date

2026-02-10

4.43.2 Macro Definition Documentation

4.43.2.1 HEADBUFFER_CPP

```
#define HEADBUFFER_CPP
```

4.44 src/Project 3.0/TestDriver.cpp File Reference

Minimal scenario runner for Project 3.0.

```
#include <cstdlib>
#include <filesystem>
#include <iostream>
#include <sstream>
#include <string>
```

Functions

- int main (int argc, char *argv[])

  Main entry point for the lightweight scenario runner.

4.44.1 Detailed Description

Minimal scenario runner for Project 3.0.

Author

Robert Shores This executable is intentionally lightweight and delegates core behavior to the canonical production CLI in bin/main.

4.44.2 Function Documentation

4.44.2.1 main()

```
int main (
            int argc,
            char * argv[])
```

Main entry point for the lightweight scenario runner.

Parameters

	argc	Argument count.
	argv	Argument vector.

Returns

Process exit code.

4.45 src/RecordFormatter.cpp File Reference

Implements the RecordFormatter class, which provides methods for packing and unpacking ZipCode.

```
#include "RecordFormatter.h"
#include <iomanip>
```

4.45.1 Detailed Description

Implements the RecordFormatter class, which provides methods for packing and unpacking ZipCode.

Author

Gabriel Kofi Coffie Boafo & Ken Tabata

Date

2026-02-14

4.46 src/SearchIndex.cpp File Reference

Search program for blocked-sequence data using a simple primary-key index.

```
#include <iostream>
#include <string>
#include <vector>
#include "PrimaryKeyIndex.h"
```

Functions

- int main (int argc, char *argv[])

  Main entry point for indexed ZIP lookup.

4.46.1 Detailed Description

Search program for blocked-sequence data using a simple primary-key index.

Author

Zach

Date

2026-02-10

4.46.2 Function Documentation

4.46.2.1 main()

```
int main (
            int argc,
            char * argv[])
```

Main entry point for indexed ZIP lookup.

Parameters

	argc	Argument count.
	argv	Argument vector.

Returns

Process exit code.

4.47 src/ZipCodeBuffer.cpp File Reference

Implementation file for the ZipCodeBuffer class, which provides methods for reading and managing zip code data.

```
#include "../include/ZipCodeBuffer.h"
#include <stdexcept>
#include <sstream>
```

4.47.1 Detailed Description

Implementation file for the ZipCodeBuffer class, which provides methods for reading and managing zip code data.

Author

Ken Tabata & Gabriel K Coffie Boafo

Date

2026-02-11

4.48 src/ZipCodeRecord.cpp File Reference

Implementation file for the ZipCodeRecord class, which provides a data structure for storing zip code information.

```
#include "../include/ZipCodeRecord.h"
```

4.48.1 Detailed Description

Implementation file for the ZipCodeRecord class, which provides a data structure for storing zip code information.

Author

Ken Tabata & Gabriel K.Coffie Boafo

Date

2026-02-12